

Better CodeLlama: A LLM Compiler for Translating C Code to x86 Assembly

Columbia COMS4705 Project

Keywords: *compiler, fine-tuning, CodeLlama, machine translation, C, x86, SFT, GRPO, PPO, Reinforcement Learning*

Song Li

Department of Computer Science
Columbia University
s15807@columbia.edu

Aziz Zeynalli

Department of Computer Science
Columbia University
agz2115@columbia.edu

Abstract

We fine-tuned CodeLlama-7B-Instruct and further compare two reinforcement learning methods to translate C code into x86 assembly. We have explored two instruction-tuning strategies: SFT and SFT with few-shot examples. Both approaches improved significantly over the original, even including the internal mathematical ability to calculate stack frame memory addresses. Our fine-tuned model generates syntactically correct assembly with proper prologues, register usage, control flow, functionality, epilogues, etc. Further, we refined our models by incorporating two reinforcement learning techniques: GRPO and PPO, targeting better compilation rate and semantics. Our result proves the potential of language model's capability of convoluted compiler process, and policy optimization based reinforcement learning on binary compilation rate is effective for language models to learn to directly produce successful compilable assembly code.

1 Key Information to include

- Mentor: Joey Huang
- External Collaborators (if you have any): No
- Sharing project: No

2 Introduction

Relatively small language models with less than 10 billion parameters have already proven to be capable in the mathematical-reasoning realm. For software engineering, one challenging task that requires advanced mathematical abilities is the code compilation, such as the memory allocations, control flows, or even intermediate code optimizations. Traditionally, developers utilize the GNU Compiler Collection (GCC) to compile the programmed C code to assembly code for the target machines. However, with the boom of AI agentic software development, the language models are likely to hallucinate and generate uncompileable C codes, and the cost to repair incorrect codes and recompilation can be significant.

In this study, we built a better model empowered with sufficient end-to-end compilation abilities, which is able to generate compilable and correct assembly code straight from C code. One promising use-case of our model includes memory-related cyber-security enhancement, and our model is able to provide more contexts on current function calling stack frame memory layout in real time development, and it reduces the workload for software developers to inspect and check for vulnerabilities in real time while they are coding with our AI. We believe that our work can serve as a leap forward for the research community to incorporate language model's potential even into low-level languages

and design AI agents for modern compiler to deal with complex tasks such as inter-procedural code optimization in compilation. Current commercial AI agents simply use function calling to invoke gcc to generate desired compiled code, but this tool-using approach does not reveal a model’s intrinsic understanding on compilers.

We chose to initialize our base model with CodeLlama-7B-Instruct [1], since we noticed that training a model, specialized for coding, is a better choice compared to training from a general-purpose LLM for our code-generation need.

We collected over 15K C files from open-source projects, and published our curated dataset to HuggingFace ¹. We apply instruction-tuning on top of this foundation model, and the tuned model outperforms the base model significantly.

Furthermore, we used two reinforcement learning methods: the novel Group Relative Policy Optimization (GRPO) [2] and the traditional Proximal Policy Optimization (PPO) [3] to strengthen the performance of the instruction-tuned model. We applied two techniques separately on top of our tuned model, and both models after two RL approaches exhibit considerable compilation rate progress with as little as 300 training examples (compilation rate: SFT 0.38 $\rightarrow$ GRPO 0.48 and PPO 0.46). This also additively implies that language models are able to internally generate correct reasoning on all compilation stages, including lexical, parsing, semantic analysis, etc. to generate completely compilable assembly codes. Our models even surpass popular closed-source GPT-4o-mini [4] from OpenAI in producing compilable assembly code.

3 Related Work

Large language models have shown strong performance on code generation tasks. CodeLlama [1] by Meta AI demonstrated that continued pretraining on code corpora enables models to generate functionally correct programs across multiple languages. Most work focuses on high-level language translation (e.g., Python to Java) with limited exploration of high-level to low-level code generation. Meta AI’s work was limited to low-level language to optimized low-level language. A huge leap of translation from high-level language to assembly code remains to be a possibility for testing language model’s abilities.

To maximize specialized reasoning abilities in a language model, DeepSeek-AI [5] emphasized the use of reinforcement learning, providing an insight to allow the model to self-evolve based on their own generations through reward mechanism. This indicates the potential of improving reasoning abilities without any supervised data. Especially in the field that requires extensive mathematical logical reasoning, simple reward signals with exclusively binary accuracy feedback are impressive in the RL training. DeepSeek-AI oriented their work primarily on general-purpose coding and mathematical reasoning tasks, but the potential of GRPO [2] has yet to be explored in the similar mathematically driven task: compilation.

Cummins et al. [6] introduced the Meta LLM Compiler, a suite of models pretrained on 546 billion tokens of LLVM-IR and assembly code for compiler optimization tasks. Their work demonstrates that LLMs can perform backend optimization, compiler emulation, and disassembly at high proficiency. However, their models target LLVM-IR level optimization rather than frontend compilation from high-level languages like C to assembly. LLM4Decompile [7] fine-tuned models up to 33B parameters specifically for decompilation (assembly to C), achieving 21-22% re-executability. While decompilation has received attention, the forward compilation direction from C to assembly remains underexplored.

Recent work has applied reinforcement learning to improve code generation quality. Policy optimization methods like PPO [3] and GRPO [2] have been used to optimize for execution-based rewards. We adapt these RL approaches specifically for binary compilation success as a reward signal.

4 Approach

We employed a multi-stage training pipeline. First, we fine-tuned CodeLlama-7B-Instruct using Supervised Fine-Tuning (SFT) with Low-Rank Adaptation (LoRA) [8] on a curated dataset of C-

¹HuggingFace Dataset ID: AzizZeynalli/c2x86-dataset

to-Assembly pairs. This stage ensures the model learns the syntax and structure of x86 assembly. Subsequently, we applied Group Relative Policy Optimization (GRPO) to further optimize the model for compilation success and code similarity. GRPO uses a group of generated outputs to estimate the baseline, eliminating the need for a separate value network. Additionally, we included PPO implementation into our training stages to observe the key differences of two RL methods and their own benefits.

4.1 Baselines

We evaluate against three baselines. First, CodeLlama-7B-Instruct [1] without any fine-tuning serves as our primary baseline, representing the out-of-the-box capability of a code-specialized LLM on the C-to-assembly task. CodeLlama is selected because it is pre-trained on 500B tokens of code from GitHub across multiple programming languages, providing a stronger starting point for compilation tasks than general-purpose LLMs. We prompt it with the same system instruction and zero-shot format used for our fine-tuned models. Second, we include GPT-4o-mini and GPT-4o [9] as proprietary baselines to benchmark against state-of-the-art commercial models. These models are prompted with identical instructions specifying x86-64 AT&T syntax assembly for Debian OS, with explicit guidance to omit non-essential directives. GPT-4o and GPT-4o-mini establish an upper bound for what general-purpose LLMs can achieve on this task without domain-specific fine-tuning.

4.2 Implementations

We use pipelines from HuggingFace `trl` packages to support consistent and reliable experiments. All of our SFT, GRPO, and PPO are enabled by corresponding trainer packages in `trl`.

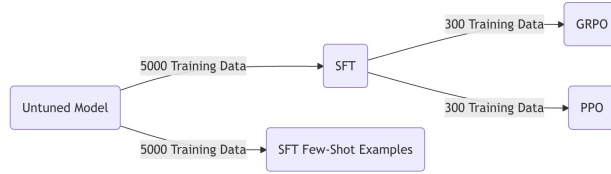


Figure 1: Post-training Pipeline.

5 Experiments

5.1 Data

We curated our dataset from 15,000 standalone C source files with no external library dependencies and their corresponding x86 assembly code compiled with GCC on Debian OS. The majority data are short contest solutions, tutorials, interviews, textbooks algorithms, etc. The dataset was sanitized to remove comments and non-essential directives, shuffled, and separated into train-test splits. Our dataset consists of 80% of training split with around 8k pairs of C codes and their corresponding assembly codes and 20% of validation split with 2k pairs. All of data are manually sampled from publicly available but small GitHub repositories not from large open HuggingFace datasets, and this should prevent data pollution so that the models have not seen the same data in their pre-training stages.

The dataset selection aligns with our goal to reflect real AI agent’s behavior while working with large code bases. The LLMs typically concentrate on each file to narrow down the scope and not to lose the attention across multiple dependent files. The sources of data are expected to be diverse and inclusive on every possible operations.

For SFT, we randomly sample 5,000 training pairs. For GRPO, we used a separate set of randomly sampled 300 training pairs. For evaluation, we randomly sample 200 pairs from our test split of the dataset.

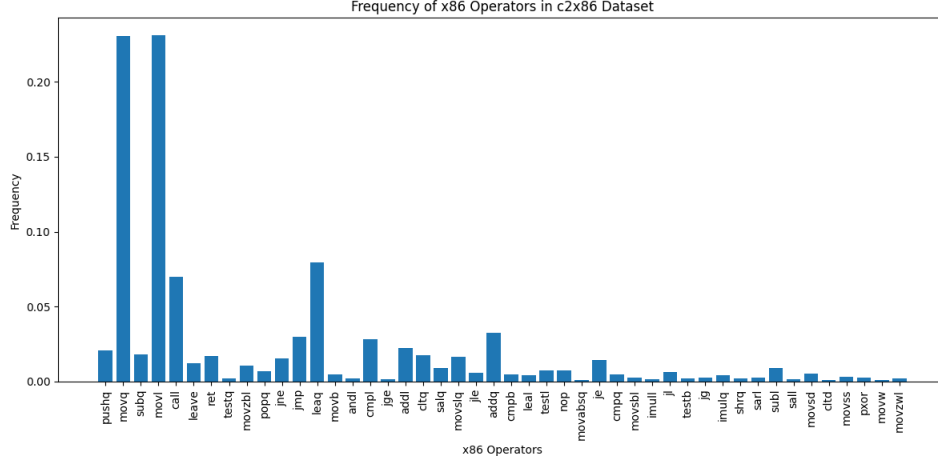


Figure 2: Our tuned models manifest strong performances on generating data movement and control flow related operations.

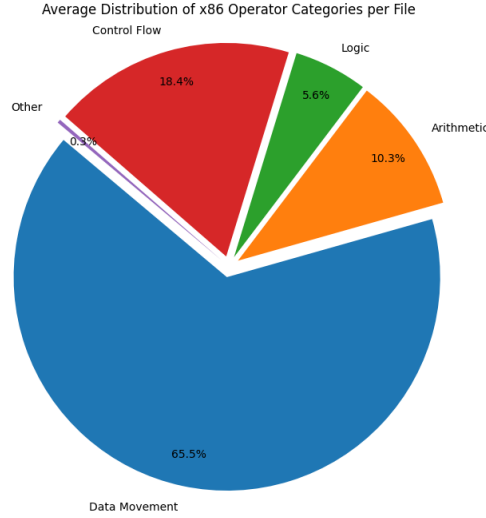


Figure 3: Our curated data simulate the real-world programs with the reasonable frequency of each operator category. This pie-chart reflects typical C program structure with heavy register/memory operations and moderate branching.

5.2 Evaluation method

We have four evaluation metrics ordered by the importance:

1. **Compilation Success Rate:** Checks if generated assembly can be further compiled to machine code using GCC.
2. **Length of Longest Common Subsequence (LCS):** Capture the longest common subsequence between generated and target assembly codes.

$$\text{normalized LCS} = \frac{\text{LCS}}{\text{Length of Target Code}} \quad (1)$$

3. **Levenshtein Distance (LEV):** Measures minimum edits needed to transform generated codes to target codes.

$$\text{normalized LEV} = \frac{\text{LEV}}{\text{Length of Target Code}} \quad (2)$$

4. **Functionality Check:** Verifies if the generated reproduces original C code behavior using 20 curated leetcode test ² cases with main functions to verify the functionality test results.

Compilation Success Rate measures whether the model produces assembly that can be compiled into executable machine code, capturing practical viability. Length of Longest Common Subsequence (LCS) quantifies structural similarity to the target assembly, reflecting correct ordering and syntax. Assembly codes are sequentially written, and it is not a typical behavior for modern compilers even to reorder the control flows. Also the consistent naming convention makes evaluation on LCS extraordinarily effective. Normalized Levenshtein Distance (LEV) captures the edit distance between generated and gold code, indicating overall accuracy. Functionality Check verifies that the compiled output reproduces the original program’s behavior on a suite of tests, ensuring semantic fidelity. Together, these metrics evaluate practical correctness, structural fidelity, and functional equivalence.

5.3 Experimental details

Training Parameters	SFT	Few-shot	GRPO	PPO
Batch Size	2	2	4	16
Grad. Acc. Steps	4	4	2	16
Learning Rate	2e-4	2e-4	5e-5	1e-6
Number of Epochs	1	1	1	1
Precision	fp16	fp16	bf16	4-bit NF4
Optimizer	pAdamW8bit	pAdamW8bit	Adafactor	pAdamW32bit
Temperature	N/A	N/A	1.0	1.0
top_k	N/A	N/A	50	0.0
top_p	N/A	N/A	0.95	0.9
Clip Range (ϵ)	N/A	N/A	0.1	0.1
Training Data Size	5000	5000	300	300
LoRA Rank (r)	32	32	32	32
GPU	L4	A100	A100	A100

Table 1: Training parameters. Few-Shot refers to SFT trained with multiple in-context examples. N/A indicates parameters not applicable to that training method. Both pAdamW8bit and pAdamW32bit refer to paged Adam Optimizers.

5.4 Results

Our SFT model achieved a compilation success rate of 38%, a significant improvement over the base model. GRPO further improved this to 48%. In comparison, GPT-4o-mini, while powerful, often failed to strictly adhere to the requested format.

- Specialized fine-tuning is more effective than general-purpose prompting for low-level code generation.
- Few-shot SFT³ underperforms single-example SFT (0.30 vs 0.38 compile rate). The few-shot approach prepends 3 randomly sampled C-to-assembly pairs to each training prompt. This introduces two issues: (1) different random examples per training sample create inconsistent learning signal, and (2) the combined prompt is truncated at 4096 tokens, potentially cutting off the target code. The longer context from unrelated examples may also confuse the model rather than help it generalize.
- The improvement with GRPO underpins the effectiveness of reinforcement learning in optimizing for binary compilation success.
- PPO achieves comparable performance to GRPO (compile rate 0.46 vs 0.48), suggesting that both policy optimization approaches effectively leverage the binary compilation reward. PPO’s explicit value network provides stable training at the cost of additional memory overhead.

²HuggingFace Dataset ID: SongTonyLi/c2x86-leetcode-eval

³HuggingFace ID: AzizZeynalli/codellama-4bit-peft-lora-r32-sft-c2x86-fewshot

- All models, including advanced OpenAI models, achieve 0% on LeetCode functionality tests. These tests require preserving algorithmic correctness across compilation. The gap suggests models learn syntactic patterns but fail to maintain computational equivalence for complex algorithms.

See Appendix for generation examples across all models.

Metric	Llama	4o-mini	4o	SFT	Few-shot	GRPO	PPO
Norm Levenshtein ↓	0.901	1.527	0.655	0.736	0.740	0.565	0.564
Norm LCS ↑	0.189	0.446	0.568	0.270	0.267	0.441	0.445
Compile Rate ↑	0.020	0.320	0.660	0.380	0.300	0.48	0.46
LeetCode Functionality ↑	0	0	0	0	0	0	0

Table 2: Evaluation results on 50 validation samples. ↓ lower is better, ↑ higher is better. Llama refers to the untuned base CodeLlama model and Few-Shot is SFT trained with multiple examples. 4o-mini and 4o refer to OpenAI’s proprietary models GPT-4o-mini and GPT-4o [9] respectively.

6 Analysis

6.1 SFT

Instruction tuning with SFT and LoRa is the first step to ensure the basic LLM generated code quality. In our experiment (see Appendix), CodeLlama-7B-Instruct hallucinates starting from the first line. For example, in our input C code we do not provide the file name, but the generated assembly code creates a non-essential directive file name. This behavior is also observed in OpenAI’s GPT-4o-mini. Besides the non-essential directives, the base model also generates non-standard x86 assembly codes along with incorrect memory addresses. One incorrect line in the assembly code can lead to incorrect compilation. SFT is crucial for adapting the general-purpose coding model to the specific task of translation. It aligns the model’s output with the expected assembly format, reducing hallucinations and syntax errors. LoRA allows for efficient fine-tuning of large language models by updating only a small subset of parameters, significantly reducing memory requirements and training time while maintaining performance. We published our trained model publicly on HuggingFace ⁴.

6.2 GRPO

Reinforcement learning with GRPO is essential for tasks with clear quantitatively calculable objectives. For example, mathematical solving tasks can be measured based on binary numerical correctness metric. After the instruction tuning, despite the significant improvement in the normalized LCS compared to target codes, only 38% of the generated assembly codes are further compilable to machine code on Debian OS with x86 architecture. Binary compilation success rate is a perfect candidate for GRPO, and with the restraints on clip-rate, GRPO is able to raise the generated code compilability without deviating too much from our instruction-tuned model.

In our experiment, GRPO produces four candidate generations for every single compilation request by allowing sampling and setting the temperature to 1. Our greedy approach is sufficient to generate good-quality assembly code, and we discovered that adjusting model temperature to 1 is able to allow us to amplify our reward signal with samples in the validation dataset, where α is a hyper-parameter to be experimented with to magnify the reward signal variance across all generation examples. We verified that setting the temperature to 1.0 produced sufficient diversity in the generated samples, which is critical for GRPO to estimate the advantage function effectively.

$$\text{reward signal} = \alpha * \text{binary compilation success} + (1 - \alpha) * \text{normalized LCS} \quad (3)$$

GRPO uses advantage normalization. If all generations for a prompt have similar rewards, i.e. low reward variance, the advantages become nearly zero, leading to zero policy gradient loss. GRPO is not able to update the policy in one iteration, because it can not decide which generation to learn

⁴HuggingFace ID: SongTonyLi/codellama-4bit-peft-lora-r32-sft-c2x86

from. In our experiment, we choose to compare 4 candidate generations per iteration in our trainer, because our instruction-tuned model displays over 38% compilation success rate.

Using this reward signal, our RL approach prioritizes the successful generation able to be compiled, and if all generations succeed or fail, GRPO can choose the one most similar to the target code to learn on. One clear benefit of using GRPO is the outcome-supervised RL: outcome supervision sets the advantages of all tokens in every generation as the normalized reward to update the policy. Only the highly rewarded generation is effective in updating the base model. During our exploration, we fixed our clip ratio, i.e ϵ , in our trainer configuration to avoid excessive deviation from our tuned model while boosting compilation rate steadily. We published our trained model publicly on HuggingFace ⁵.

6.3 PPO

Proximal Policy Optimization (PPO) is a policy gradient method that constrains policy updates using a clipped surrogate objective, which prevents large deviations from the reference model. Different from GRPO, PPO maintains an explicit value network to estimate advantages. This requires additional memory but can provide more stable gradient estimates.

In our experiment, we initialize PPO from the SFT-tuned model and load a frozen copy as the reference model. We set the KL coefficient to 0.1 and enable adaptive KL control with target KL of 0.1 to prevent the policy from diverging too much from our instruction-tuned baseline. Score scaling and normalization are enabled to stabilize training. We use PagedAdamW32bit optimizer with learning rate 1e-6, which is significantly lower than GRPO, to make sure updates are conservative given PPO’s separate value network.

PPO generates one candidate per prompt during training with top-p sampling ($p=0.9$) and temperature 1.0. The clipped surrogate objective with cliprange 0.1 makes sure policy updates stay conservative. We train on 300 examples with effective batch size of 256 (batch size $16 \times$ gradient accumulation 16). Due to CUDA out of memory issues from maintaining the value network, we limited max_new_tokens to 256 during training, but evaluated the model with max_new_tokens 512.

We use the same reward signal formula 3 for PPO. With $\alpha = 0.5$, PPO achieves 46% compilation rate with normalized LCS of 0.445 and normalized Levenshtein of 0.564. With $\alpha = 0.8$, PPO achieves 41% compilation rate but normalized LCS drops to 0.125 and normalized Levenshtein increases to 0.883. This shows a clear tradeoff: heavily weighting compilation success causes the model to generate code that compiles but deviates more from the target assembly structure. The $\alpha = 0.5$ configuration provides better balance between compilation success and code similarity. We published our trained model publicly on HuggingFace ⁶.

6.4 Evaluation

Our instruction-tuned model guarantees the basic compilation success rate and decent LCS scores to demonstrate its lexical, parsing, syntactical, and semantical analytical skills, but only partial functionality of the program is preserved. Both policy optimization methods can effectively learn to produce compilable assembly from the binary reward signal. The small difference between GRPO and PPO suggests that for this task, the choice of RL algorithm matters less than how we design the reward signal. Compared to PPO, GRPO forgoes the explicit value models and leads to faster and more intuitive training improvement. PPO, on the other hand, steadily raises the performances with well-tested algorithm.

We identify three failure patterns:

- Structural errors (missing prologues/epilogues or incorrect labels) which cause compilation failure (common in base model).
- Arithmetic errors (operations like $i * 2$ incorrectly generated as $i + 1$) which produce compilable but semantically wrong code (observed in SFT/GRPO, see Appendix Example 1).
- Addressing errors (hardcoded memory offsets instead of scaled indexed addressing) which produce incorrect memory access patterns.

⁵HuggingFace ID: SongTonyLi/codellama-grpo-clip-0.1-peft-lora-r32-c2x86

⁶HuggingFace ID: AzizZeynali/codellama-7b-c2x86-ppo-rl

Model Generation Patterns (See Appendix Example 2):

- **Architecture & ABI Compliance:** Base and GPT-4o-mini generate incorrect 32-bit x86 (`pushl %ebp, %esp`) instead of x86-64. SFT/GRPO/PPO correctly emit 64-bit instructions (`pushq %rbp, %rsp`). This fundamental architectural mismatch causes Base/GPT-4o-mini compilation failures, while fine-tuned models achieve 100% compilation success. This illustrates that early on instruction tuning stage the model learned on how to follow x86 ABI compliance, and even small instruction-tuned models exhibit huge advantage even against commercial models with simple prompting.
- **Semantic and Arithmetic Capability Advance:** GPT-4o-mini treats assembly as template-filling, missing pointer indirection semantics. SFT learns surface patterns but occasionally misses byte-scaling. RL reward signals (compilation success + execution correctness) force GRPO/PPO to discover that the correct pointer arithmetic operations.

In addition, we sampled challenging questions from LeetCode, typically designed for software engineering interviews. Solutions of LeetCode questions are expected to be algorithmically complicated. All of our models including unfine-tuned base models and OpenAI models are not able to pass the 20 LeetCode functionality checks. This is as expected, because we did not introduce secondary functionality pass rate as a reward signal in our second round RL due to limited compute resources for additional training.

The potential reason behind this is the LLM’s inability to incorporate high-level algorithmic thinking with low-level memory-related inferences. It is possible to resolve the issues by including the algorithmic functionality check into our RL reward signal. However, those models are able to generate functional codes in high-level languages, and we suspect that the fundamental underlying problem is better to be resolved by the pre-training with more assembly codes instead of post-training methods. This leads to potential future approach by task separation to invoke model’s deep thinking process by allowing the models to reason about the intents of program first and then produce compiled code with this extra information in context.

7 Conclusions

Our experiments confirm that instruction-tuning CodeLlama-7B-Instruct with LoRA tremendously improve compilability by following x86 syntax and ABI conventions. Applying policy optimization further elevates the compilation success rate, and LCS significantly, while maintaining semantic fidelity to reference GCC outputs. Nevertheless, all variants display a gap between low-level correctness and high-level algorithmic reasoning. Future work will augment the training corpus, inject a functionality reward, and explore larger backbone models. We aim to dive into the exact issues that cause the functionality failures, by enabling multi-round thinking process for the models to reason their work. Our work can be a stepping stone for the research community to utilize our findings to further fine-tune language models with more fine-grained objective in RL, such as enhancement in particular arithmetic, control flow, logical instructions.

8 Team Contributions

Song: led the entire project, designed the post-training pipelines, curated the datasets, implemented the SFT with LoRA, GRPO, and OpenAI models evaluations.

Aziz: implemented PPO training pipeline and SFT with few-shot examples, curated the C to x86 assembly dataset, collected LeetCode questions and test cases into HuggingFace dataset for functionality evaluation.

References

- [1] Baptiste Rozière et al. Codellama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- [2] Zhu Shao, Wang et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arxiv:2402.03300*, 2024.

- [3] Wolski Schulman et al. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [4] OpenAI. Gpt-4o mini: advancing cost-efficient intelligence. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>, 2024.
- [5] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. <https://arxiv.org/pdf/2501.12948>, 2025.
- [6] Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Rozière, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. Meta large language model compiler: Foundation models of compiler optimization. *arXiv preprint arXiv:2407.02524*, 2024.
- [7] Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. Llm4decompile: Decompiling binary code with large language models. *arXiv preprint arXiv:2403.05286*, 2024.
- [8] Edward J Hu et al. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [9] OpenAI. Gpt-4o system card. <https://openai.com/index/gpt-4o-system-card/>, 2024.

A Appendix

Generation Examples

We present two examples comparing outputs across all models, highlighting key differences.

Example 1: Nested Loops with Arithmetic

C code:

```
#include <stdio.h>
int main() {
    int t;
    scanf("%d",&t);
    while(t--){
        int n;
        scanf("%d",&n);
        for(int i=1; i<=n; i++){
            printf("%d ", i*2);
        }
        printf("\n");
    }
    return 0;
}
```

Base (wrong architecture, truncated):

```
.file "test.c"
.text
.globl main
.type main, @function
main:
    pushl    %ebp                ; 32-bit, should be pushq
    movl     %esp, %ebp          ; 32-bit, should be movq
    subl     $24, %esp
    movl     $0, -4(%ebp)
    movl     $0, -8(%ebp)
    ...                          ; repetitive zero initialization
```

Target (correct i*2):

```
.L4:
    movl    -4(%rbp), %eax
    addl    %eax, %eax      ; i*2 = i + i (correct)
    movl    %eax, %esi
    ...
```

SFT/GRPO (semantic bug):

```
.L4:
    movl    -12(%rbp), %eax
    addl    $1, %eax        ; i+1, NOT i*2 (incorrect)
    movl    %eax, %esi
    ...
```

PPO (correct):

```
.L4:
    movl    -12(%rbp), %eax
    addl    %eax, %eax      ; i*2 = i + i (correct)
    movl    %eax, %esi
    ...
```

Example 2: Simple Pointer Operations

C code:

```
#include <stdio.h>
int main(void) {
    int n;
    int a[5];
    int *p;
    a[2] = 1024;
    p = &n;
    *(p + 5) = 98;
    printf("a[2] = %d\n", a[2]);
    return (0);
}
```

Target:

```
.text .section .rodata
.LC0:
    .string "a[2] = %d\n"
    .text .globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $32, %rsp
    movl $1024, -24(%rbp)
    leaq -12(%rbp), %rax
    movq %rax, -8(%rbp)
    movq -8(%rbp), %rax
    addq $20, %rax
    movl $98, (%rax)
    movl -24(%rbp), %eax
    movl %eax, %esi
    leaq .LC0(%rip), %rax
    movq %rax, %rdi
    movl $0, %eax
    call printf@PLT
    movl $0, %eax
```

```

        leave
        ret
.section .note.GNU-stack,"",@progbits

```

Base (compiles=False, LEV_norm=0.784, LCS_norm=0.449):

```

.section      .data
a:
    .int 0, 0, 1024, 0, 0
.section .text .global main
main:
    pushl    %ebp
    movl     %esp, %ebp
    subl     $4, %esp
    movl     $0, -4(%ebp)
    movl     $0, -8(%ebp)
    movl     $1024, 8(%ebp)
    movl     $0, 12(%ebp)
    movl     $0, 16(%ebp)
    movl     $0, 20(%ebp)
    movl     $0, 24(%ebp)
    movl     $0, 28(%ebp)
    movl     $0, 32(%ebp)
    movl     $0, 36(%ebp)
    movl     $0,
    ...

```

GPT-4o-mini (compiles=False, LEV_norm=0.857, LCS_norm=0.508):

```

.section .bss n: .skip 4 a: .skip 20 p: .skip 4
.section .text .globl main
main:
    push %ebp
    mov %esp, %ebp
    sub $0x14, %esp
    movl $1024, a + 8(%ebp) # a[2] = 1024
    lea n(%ebp), %eax # p = &n
    mov %eax, p(%ebp)
    movl $98, 20(%ebp) # *(p + 5) = 98
    movl a + 8(%ebp), %eax # load a[2]
    push %eax
    push $.LC0 # format string call printf
    add $0x8, %esp
    movl $0, %eax
    leave
    ret
.section .rodata .LC0: .string "a[2] = %d\n"

```

SFT (compiles=True, LEV_norm=0.109, LCS_norm=0.893):

```

.text .section .rodata
.LC0:
.string "a[2] = %d\n"
.text
.globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $32, %rsp
    movl $1024, -16(%rbp)

```

```

leaq -12(%rbp), %rax
movq %rax, -8(%rbp)
movq -8(%rbp), %rax
addq $5, %rax
movl $98, (%rax)
movl -16(%rbp), %eax
movl %eax, %esi
leaq .LC0(%rip), %rax
movq %rax, %rdi
movl $0, %eax
call printf@PLT
movl $0, %eax
leave
ret

```

GRPO & PPO (compiles=True, LEV_norm=0.017, LCS_norm=0.986):

```

.text .section .rodata
.LC0:
    .string "a[2] = %d\n"
    .text .globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $32, %rsp
    movl $1024, -16(%rbp)
    leaq -12(%rbp), %rax
    movq %rax, -8(%rbp)
    movq -8(%rbp), %rax
    addq $5, %rax
    movl $98, (%rax)
    movl -16(%rbp), %eax
    movl %eax, %esi
    leaq .LC0(%rip), %rax
    movq %rax, %rdi
    movl $0, %eax
    call printf@PLT
    movl $0, %eax
    leave
    ret
.section .note.GNU-stack,"",@progbits

```

Observation:

SFT, GRPO, PPO generated very similar but correct results, and all of them are close to the target codes. Despite some numeric differences on memory addresses, the internal algorithmic logics are the same. This showcases that RL methods can avoid performance degradation from instruction tuning with proper clipping ratio. The Target code uses `addq $20, %rax` (5 integers $\times$ 4 bytes). SFT/GRPO/PPO emit `addq $5, %rax`—syntactically different but semantically equivalent given stack layout differences. GPT-4o-mini incorrectly hardcodes `movl $98, 20(%ebp)`, bypassing pointer indirection entirely and breaking the program’s intended memory aliasing behavior.

GRPO Reward Mechanism Examples

We illustrate how GRPO selects the generation to learn from, using the validation data examples. All assembly codes are truncated and reformatted for clarity.

Target:

```

.text .globl gcd
gcd:

```

```

    pushq %rbp
    movq %rsp, %rbp
    movq %rdi, -24(%rbp)
    movq %rsi, -32(%rbp)
    ...

```

Greedy: compiles = True, LCS = 0.1171

```

.text .globl gcd
gcd:
    pushq %rbp
    movq %rsp, %rbp
    movq %rdi, -8(%rbp)
    movq %rsi, -16(%rbp)
    ...

```

Sample 0: compiles = False, LCS = 0.1144, reward = 0.0572

```

.text .globl gcd
gcd:
    pushq %rbp
    movq %rsp, %rbp
    movq %rdi, -24(%rbp)
    movq %rsi, -32(%rbp)
    ...

```

Sample 1: compiles = True, LCS = 0.1240, reward = 0.5620

```

.text .globl gcd
gcd:
    pushq %rbp
    movq %rsp, %rbp
    subq $32, %rsp
    movq %rdi, -8(%rbp)
    ...

```

Sample 2: compiles = False, LCS = 0.1207, reward = 0.0603

```

.text
gcd:
    pushq %rbp
    movq %rsp, %rbp
    movq %rdi, -24(%rbp)
    movq %rsi, -32(%rbp)
    ...

```

Sample 3: compiles = True, LCS = 0.1153, reward = 0.5576

```

.text .globl gcd
gcd:
    pushq %rbp
    movq %rsp, %rbp
    movq %rdi, -24(%rbp)
    movq %rsi, -32(%rbp)
    ...

```

Observation:

Our model creates 4 generations. The standard deviation across all generated examples' rewards, 0.2505, is large enough, showing that our model's outputs are diverse and it can incentivize on the most successful results with the best answer.